

KT SESSION · TEJSWITA · OCTOBER 2026

jauth signs every FixHomi user in.

A Spring Boot service that owns identity: the **users** table, passwords, OTPs, Google and Apple sign-in, and the tokens that the mobile app, the Node backend, the admin panel and the website all trust.

50 endpoints	9 controllers	14 tables	5 roles	24 h access token	60 d refresh token
------------------------	-------------------------	---------------------	-------------------	-----------------------------	------------------------------

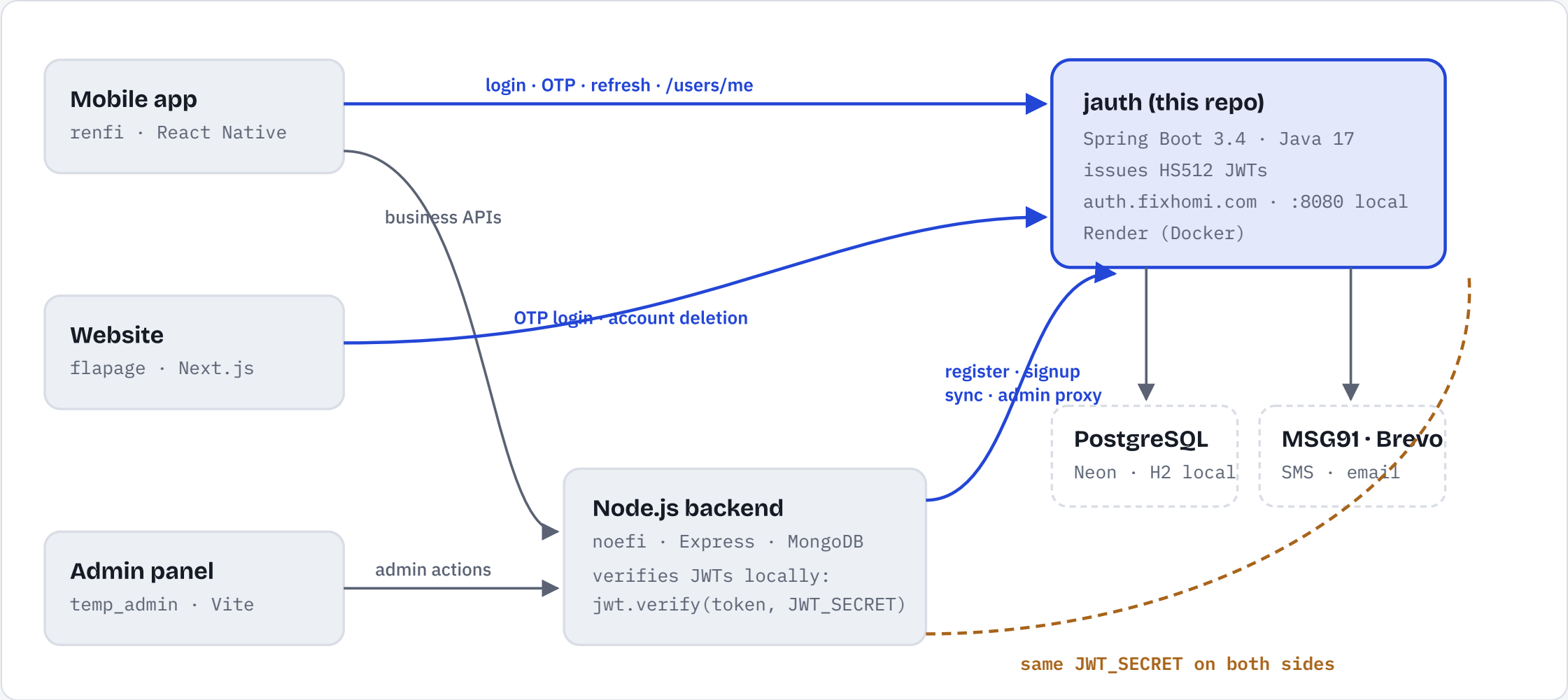
jauth owns - Accounts: email, phone, password hash, role, active/verified flags - Every way to prove identity: password, phone OTP, email OTP, phone signup, Google, Apple - Issuing and rotating tokens; password reset; account deletion - Role-based access to its own admin API	jauth does not own - Profiles, addresses, bookings, payments, service requests - Those live in the Node.js backend and MongoDB - The link between the two: <code>Mongo _id = jauth users.id</code>
---	--

01 What jauth is	02 Architecture	03 Inside the service	04 Tokens & roles	05 One request, end to end	06 Run it locally	07 Your reading list
----------------------------	---------------------------	---------------------------------	-----------------------------	--------------------------------------	-----------------------------	--------------------------------

SAY Your first task touches the SUPPORT role. The role already exists, but today it grants nothing beyond a normal user. By slide 4 you'll know exactly where roles are checked.

ARCHITECTURE

jauth signs tokens. Everyone else trusts the signature.



Trust model

jauth signs tokens with `JWT_SECRET`. Node holds the same secret and checks every token itself, with **no call to jauth** per request. If the two secrets differ, every Node request fails.

Who calls what

The app calls jauth directly for identity and Node for business. The admin panel only talks to Node, which forwards the admin's token to `/api/admin/users/**`.

Environments

auth.fixhomi.com ↔ api.fixhomi.com in prod.
jauth-1 ↔ noefix-1-dev on Render for staging.
localhost:8080 ↔ :5001 on your laptop.

SAY Signup goes through Node because Node must also create the Mongo profile with the same id.
Docs: 02 and 07.

INSIDE THE SERVICE

Every request takes the same path.

01 · Filter JwtAuthenticationFilter Bearer token → user id + ROLE_x. Checks the user still exists and is active.	02 · Rules SecurityConfig Public list, /api/admin/** = ADMIN or IT_ADMIN, everything else needs a token.	03 · Filter RateLimitingFilter Per IP: 10/min login, 5/min OTP, 100/min other.	04 · Web Controller @Valid request DTO, reads the caller's id from the security context.	05 · Logic Service @Transactional business rules; issues tokens; sends SMS/email.	06 · Data Repository → Entity Spring Data JPA on PostgreSQL (prod) or H2 (local).	07 · Errors GlobalExceptionHandler Exceptions → HTTP status + ErrorResponse JSON.
--	--	--	--	---	---	---

Package map: com.fixhomi.auth

config/ SecurityConfig, rate limiter, SMS/email wiring, local admin seeder	controller/ 9 REST controllers, 50 endpoints	dto/ request/response classes with validation	service/ one service per flow + notification/
security/ JwtService, JwtAuthenticationFilter, OAuth2 handlers, TokenHasher	entity/ 14 JPA entities + the Role enum	repository/ 13 Spring Data interfaces	exception/ custom exceptions + GlobalExceptionHandler
Controllers by URL /api/auth Auth, OtpLogin, PhoneSignup, OAuth2, Verification, Session · /api/token Token · /api/users User (own account) · /api/admin/users AdminUser.		Profiles local file H2, OTPs printed to the console, seeded admin · test for ./mvnw test · prod Neon PostgreSQL, MSG91, Brevo on Render.	

SAY Spring Security runs before the rate limiter, so a request without a valid token never consumes rate-limit budget. Docs: 02 and 04.

TOKENS & ROLES

The role travels inside the signed token.

Access token (HS512 JWT, 24 h)

```
// header
{"alg": "HS512"}
// payload – readable by anyone, signed by jauth
{
  "userId": 13,           // users.id = Mongo _id
  "role": "USER",         // becomes ROLE_USER
  "tokenType": "ACCESS",
  "sub": "walk.through@example.com",
  "iat": 1790944643, "exp": 1791031043,
  "iss": "fixhomi-auth-service"
}
```

Refresh token: opaque, 60 days, stored in `refresh_tokens`. Every refresh replaces it; replaying the old one within 45 s returns the same new one. Logout revokes the refresh token only; the access token lives until `exp`.

How a role is enforced

- `JwtAuthenticationFilter:87` builds the authority `"ROLE_" + role` from the **token**
- `SecurityConfig:113` `/api/admin/**` → `hasAnyRole("ADMIN", "IT_ADMIN")`
- `AdminController:@PreAuthorize` on all 5 methods
- `AuthService:137` `register` allows `USER / SERVICE_PROVIDER` only
- `UserService:399` `admin create` allows `ADMIN / IT_ADMIN / SUPPORT` only

SUPPORT today

Created only by an admin via `POST /api/admin/users`. Can log in with password or OTP and use its own account. Cannot reach `/api/admin/**`, is not shown in the admin user list, and is refused by Node's `requireAdmin`.

ENDPOINT GROUP	USER	PROVIDER	SUPPORT	ADMIN	IT_ADMIN
Public endpoints (register, logins, OTP, refresh...)	✓	✓	✓	✓	✓
Own account: <code>/api/users/*</code> , sessions, <code>/api/token/*</code>	✓	✓	✓	✓	✓
DELETE <code>/api/users/{id}</code> for another user	✗	✗	✗	✓	✗
<code>/api/admin/users/**</code> (create staff, list, disable, delete)	✗	✗	✗	✓	✓
Mobile Google / Apple sign-in	✓	✓	✗	✗	✗
Self-register via <code>/api/auth/register</code>	✓	✓	✗	✗	✗

SAY A denied or token-less request gets a 302 redirect to Google sign-in, not 401/403. And the role column only accepts the five roles it was created with. Docs: 04, 06 §3, 10.

ONE REQUEST, END TO END

Phone OTP login, traced through every layer.

Captured from a local run of the service. Full version with every outcome: docs/onboarding/08.

① **Send OTP** POST /api/auth/login/phone/send-otp

01 · public · no token

Request arrives

The app sends the phone number. There is no Authorization header, so JwtAuthenticationFilter sets nothing. The path is on the public list, and the rate limiter puts it in the OTP bucket (5 per minute per IP).

```
POST /api/auth/login/phone/send-otp
{"phoneNumber": "+919812345678"}
```

02 · OtpLoginController.java:78

Controller validates

@Valid checks the phone pattern ^\+?[1-9]\d{6,14}\$ before the method runs. A bad format returns 400 with validationErrors.phoneNumber.

```
@PostMapping("/phone/send-otp")
public ResponseEntity<?> sendPhoneLoginOtp(
    @Valid @RequestBody PhoneOtpLoginRequest request)
```

03 · OtpLoginService.java:98-145

Service applies the rules

Normalises to 9812345678. Allows at most 3 sends per phone per 5 minutes (else 429). The account must exist (else 404 USER_NOT_FOUND) and be active. Older unused codes are cancelled.

```
phoneNumber = User.normalizePhoneNumber(phoneNumber);
if (recent >= 3) throw new TooManyRequestsException(...);
if (user == null) throw new AuthenticationException(...);
phoneOtpRepository.invalidateAllUserOtps(user.getId(), storedPhone);
```

04 · phone_otp · SmsService

Code stored, SMS sent

6 digits from SecureRandom, saved with a 5-minute expiry. SmsService is MSG91 in production; locally StubSmsService prints the code to the console.

ID	USER_ID	PHONE_NUMBER	OTP	VERIFIED	ATTEMPTS
7	13	9812345678	652707	FALSE	0

```
StubSmsService : OTP CODE: 652707
```

05 · 200 OK

Response

The phone is masked in the response.

```
{"success":true,"maskedPhone":"****5678",
 "expiresInMinutes":5,
 "message":"OTP sent successfully to ****5678"}
```

② **Verify** POST /api/auth/login/phone/verify

06 · [OtpLoginService.java:151](#)

Check the code

Takes the newest unused code, counts the attempt (3 allowed), compares in constant time. noRollbackFor keeps the attempt count even when a wrong code throws.

```
@Transactional(noRollbackFor = VerificationException.class)
otpEntry.incrementAttempts();
if (!MessageDigest.isEqual(stored, given))
    throw new VerificationException("Invalid OTP...");
```

07 · [JwtService.java:43](#) · [RefreshTokenService](#)

Issue tokens

Marks the code used and the phone verified, updates last_login_at, signs a 24-hour JWT and stores a 60-day refresh token.

```
jwtService.generateAccessToken(user.getId(), user.getEmail(),
user.getRole());
refreshTokenService.createRefreshToken(user); // row in
refresh_tokens
```

08 · 200 OK

LoginResponse

The app stores both tokens in the Keychain and sends the access token on every call.

```
{
  "accessToken": "eyJhbGciOiJIUzUxMiJ9...",
  "refreshToken": "111e9ca9-...882f",
  "userId": 13, "role": "USER", "expiresIn": 86400,
  "isPhoneVerified": true, ...}
```

③ Use the token GET /api/token/validate

09 · [JwtAuthenticationFilter.java:45–107](#)

Filter authenticates

Verifies the HS512 signature and expiry, loads user 13 and checks it is still active (else 401 ACCOUNT_DELETED), then sets principal "13" with authority ROLE_USER.

```
new UsernamePasswordAuthenticationToken(
    String.valueOf(user.getId()), null,
    List.of(new SimpleGrantedAuthority("ROLE_" + role)));
```

10 · [TokenController.java:34](#) → [TokenValidationService](#)

Controller answers

Reads the claims back out of the token. A missing or bad token never gets this far: Spring answers with a 302.

```
{
  "valid": true, "userId": 13, "email": "walk.through@example.com",
  "role": "USER", "tokenType": "ACCESS",
  "issuedAt": 1790944643, "expiresAt": 1791031043}
```

11 · [noefi](#) · [authMiddleware.js:145](#)

How Node does the same

Node never calls this endpoint. It verifies the same token itself with the shared secret, then uses userId as the Mongo_id.

```
const decoded = jwt.verify(token, process.env.JWT_SECRET,
    { algorithms: ['HS512'] });
```

RUN IT LOCALLY

Three commands, no production credentials.

```
# JDK 17 · Maven comes with ./mvnw
cp .env.local.example .env
openssl rand -base64 64 | tr -d '\n'  # → JWT_SECRET in .env
./run.sh local

curl localhost:8080/api/auth/health
# {"status":"UP","message":"Auth service is running"}

./mvnw test                                # BUILD SUCCESS
```

The local profile gives you

- H2 database in ./localdb; delete the folder to reset
- OTPs and email links **printed to the console**
- Admin account admin@fixhomi.local / LocalAdmin@123
- DB browsing from the IntelliJ Database tool while the app runs

Postman: 14 folders, 72 requests, every endpoint

Import both files from postman/ and run the folders in order. Scripts save tokens and ids; paste each console OTP CODE into the OTP variable.

- 00 Health
- 01 Register & login
- 02 Who am I
- 03 Verify phone/email
- 04 OTP login
- 05 Phone signup
- 06 My account
- 07 Reset password
- 08 Sessions
- 09 Admin
- 10 RBAC in action
- 11 Delete
- 12 Social
- 13 Logout

Things that will surprise you

- No/expired token or wrong role → **302** to Google, not 401/403
- A stale token on a public endpoint gives 401 ACCOUNT_DELETED; send public calls with no auth
- Role changes apply only after a refresh or new login
- Logout does not kill the access token (24 h)
- Rate limits hit quickly; set RATE_LIMIT_ENABLED=false for the runner

SAY Run folders 09 and 10 live: create a SUPPORT user as admin, log in as SUPPORT, and watch the admin call get refused. Docs: 01 and 10.

What to read, in this order.

01	Local setup Run it, seeded admin, Postman, IntelliJ, troubleshooting	15 min	Day 1
02	Architecture & folder structure Where jauth fits, packages, layering, profiles	20 min	Day 2
08	Walkthrough: send OTP → validate token One request through every layer, real output	30 min	Day 2
04	Security, JWT & RBAC Filter chain, token anatomy, every role check, SUPPORT today	30 min	Day 3 · key
06	Data model 14 tables; how the role column is defined	15 min	Day 3
07	How other services use jauth App, Node, admin panel, website; the shared-secret contract	20 min	Day 4
05	Authentication flows Register, logins, signup, Google/Apple, reset, delete, admin	30 min	Day 5
10	Behaviours & gotchas Current behaviour that surprises people	15 min	Day 5
11	Working on this repo Rules, branching, where code goes, testing; plus AGENTS.md	15 min	Day 5
03 · 09	API reference · Configuration Look things up as you need them	reference	anytime

Day 1
Run it
Doc 01 end to end, then every Postman folder while watching the console.

Day 2
Understand it
Docs 02 and 08; reproduce 08 with IntelliJ breakpoints.

Day 3
Roles
Doc 04 and 06 §3; Postman folders 09–10; find the line behind each decision.

Day 4
Wider system
Doc 07; read Node's adminAuthMiddleware.js and the app's apiClient.js.

Day 5
The rest
Docs 05, 10, 11 and AGENTS.md; bring questions.

Ground rules (AGENTS.md)
This is a live production service. Check the current code, map who else is affected (Node, the app, the admin panel), flag risky changes before making them, prefer small additive steps, and verify before saying done.

Where to start
Branch docs/onboarding-tejswita → docs/onboarding/README.md. If a doc and the code disagree, the code wins. Tell Shreyash so the doc gets fixed.